PUC Minas



Aula 07 · Back-end

Banco não relacional: MongoDB Atlas + Spring Data MongoDB

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Entender **documentos**, **coleções** e o formato BSON/JSON
2. Criar um cluster **gratuito** no MongoDB Atlas
3. Criar o **usuário do banco** e liberar o **acesso de rede**
4. Conectar o Spring Boot com a *connection string* `mongodb+srv://`
5. Mapear `@Document` e usar `MongoRepository`

Antes de começar

- Aula 05: API REST
- Conta no MongoDB Atlas (login com Google ou GitHub)

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Documentos em vez de tabelas

02 Passo 1: criar o cluster

03 Passo 2: usuário do banco e acesso de rede

04 Passo 3: copiar a connection string

05 Passo 4: configurar o Spring Boot

06 Passo 5: documento e repository

07 Passo 6: ver os documentos no Atlas

08 Aprofundando: modelagem e consultas no MongoDB

01

Documentos em vez de tabelas

Documentos em vez de tabelas

No MongoDB não existem tabelas nem colunas fixas. Os dados ficam em **documentos** (parecidos com JSON) agrupados em **coleções**. Dois documentos da mesma coleção podem ter campos diferentes, e um documento pode conter listas e objetos aninhados.

Relacional (Supabase)	MongoDB
Banco	Banco (*database*)
Tabela	Coleção (*collection*)
Linha	Documento
Coluna	Campo
Chave primária `id`	Campo `_id` (`ObjectId` gerado automaticamente)
`JOIN`	Documento aninhado ou `$lookup`

Documentos em vez de tabelas

Um documento da coleção alunos

```
{
  "_id": { "$oid": "6714c0f2a1b3c45d6e7f8091" },
  "nome": "Ana Souza",
  "email": "ana@email.com",
  "curso": "Engenharia de Software",
  "disciplinas": ["DIAW", "Banco de Dados"],
  "_class": "com.example.back.model.Aluno"
}
```

Documentos em vez de tabelas

O **MongoDB Atlas** é o serviço oficial de MongoDB na nuvem. O cluster **Free** (antigo M0) tem 512 MB, é gratuito para sempre e é suficiente para todos os projetos da disciplina.

01

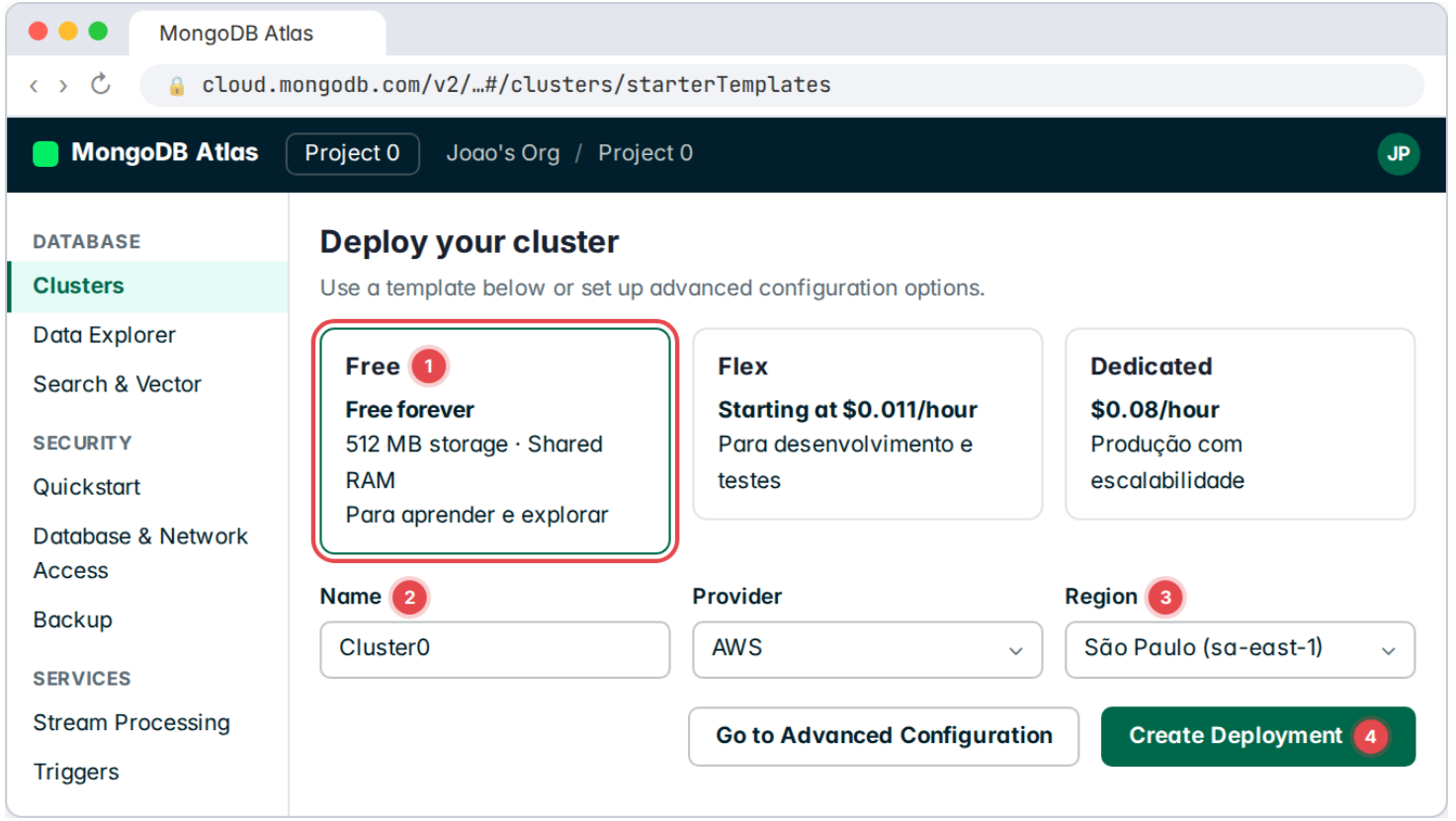
Documentos em vez de tabelas

Aula 07 · Back-end

02

Passo 1: criar o cluster

Passo 1: criar o cluster



Criação do cluster gratuito no MongoDB Atlas.

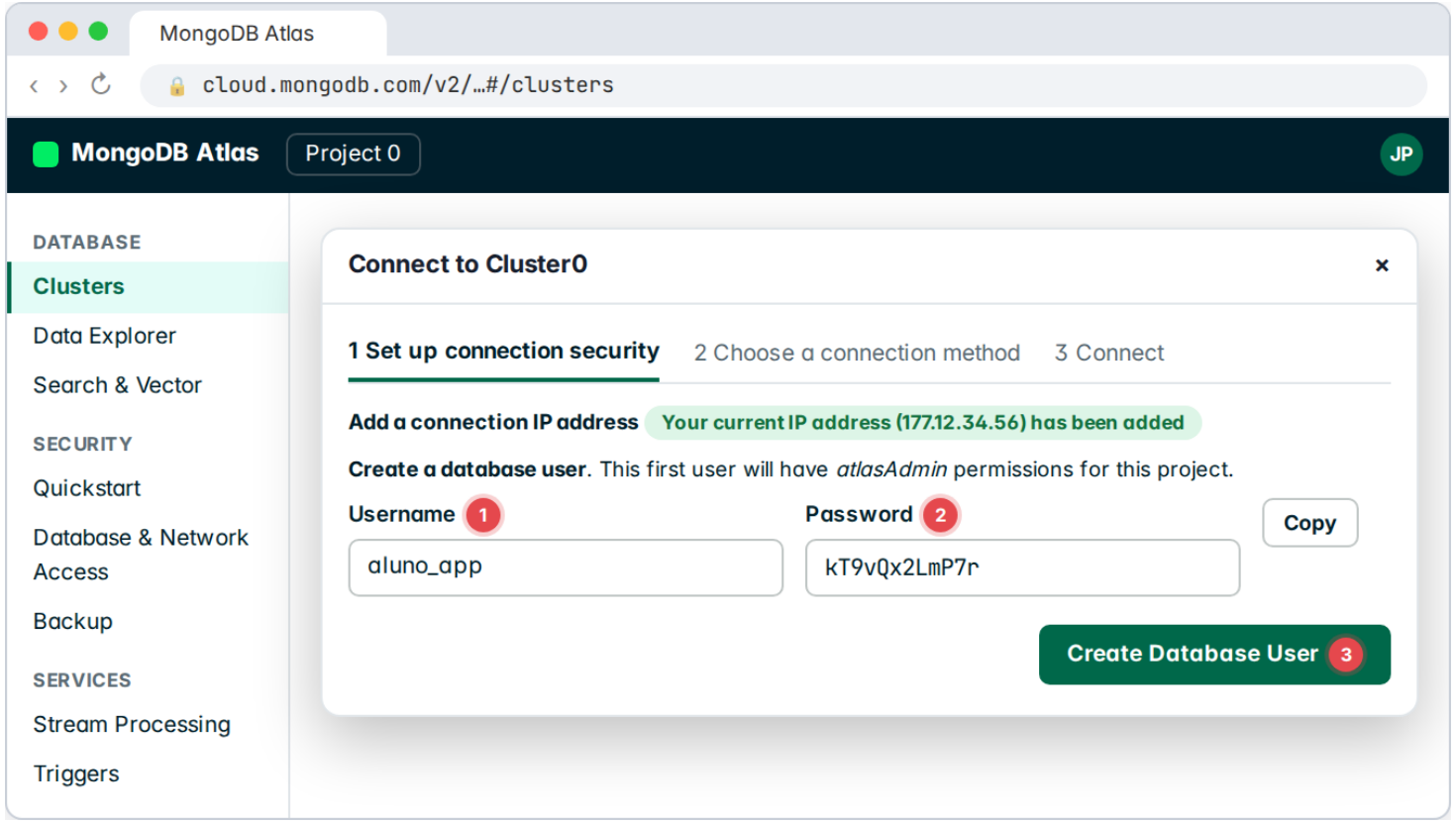
Passo 1: criar o cluster

1. Escolha **Free**.
2. Mantenha o nome `Cluster0` ou dê outro nome (ele aparece na *connection string*).
3. Provedor **AWS** e região **São Paulo (sa-east-1)**.
4. Clique em **Create Deployment**. O cluster leva de 1 a 3 minutos para ficar pronto.

03

Passo 2: usuário do banco e acesso de rede

Passo 2: usuário do banco e acesso de rede



Logo após criar o cluster, o Atlas abre o assistente **Connect to Cluster0**. Ele pede duas coisas de segurança:

Criação do primeiro usuário do banco. Prefira senhas só com letras e números.

Passo 2: usuário do banco e acesso de rede

Atenção

O usuário do banco **não é** o seu login do site do Atlas. É um usuário que só existe para a aplicação se conectar. Se a senha tiver caracteres especiais (`@ : / ? # %`), você precisará codificá-los na URL (`@` vira `%40`). Para evitar dor de cabeça, use só letras e números.

Passo 2: usuário do banco e acesso de rede

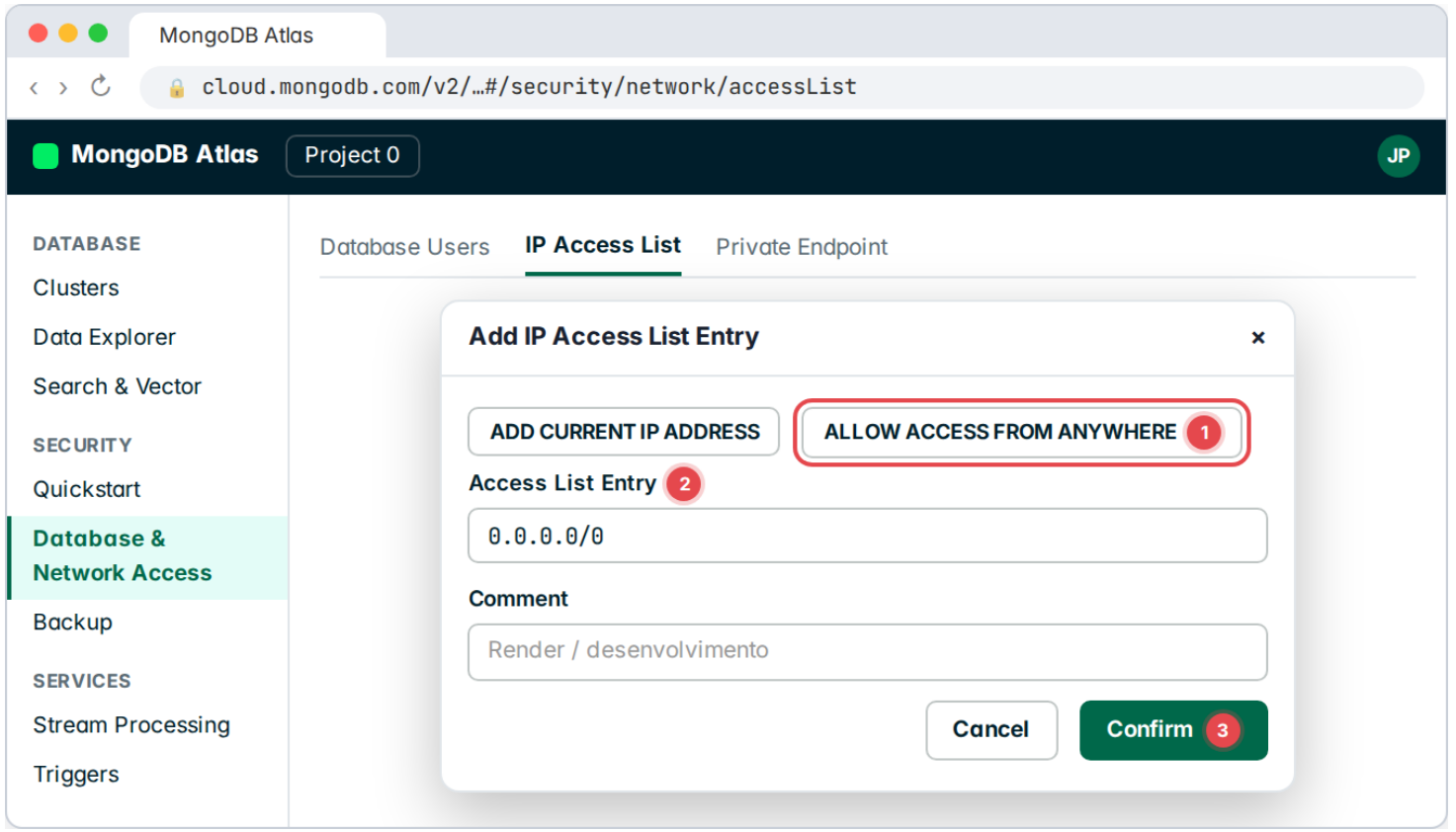
O Atlas libera apenas o **seu IP atual**. Para que o Render (que não tem IP fixo no plano gratuito) consiga conectar, libere todos os endereços em **Security → Database & Network Access → IP Access List → Add IP Address**:

03

Passo 2: usuário do banco e acesso de rede

Aula 07 · Back-end

Passo 2: usuário do banco e acesso de rede



Liberando acesso de qualquer IP (`0.0.0.0/0`), necessário para o deploy no Render.

Passo 2: usuário do banco e acesso de rede

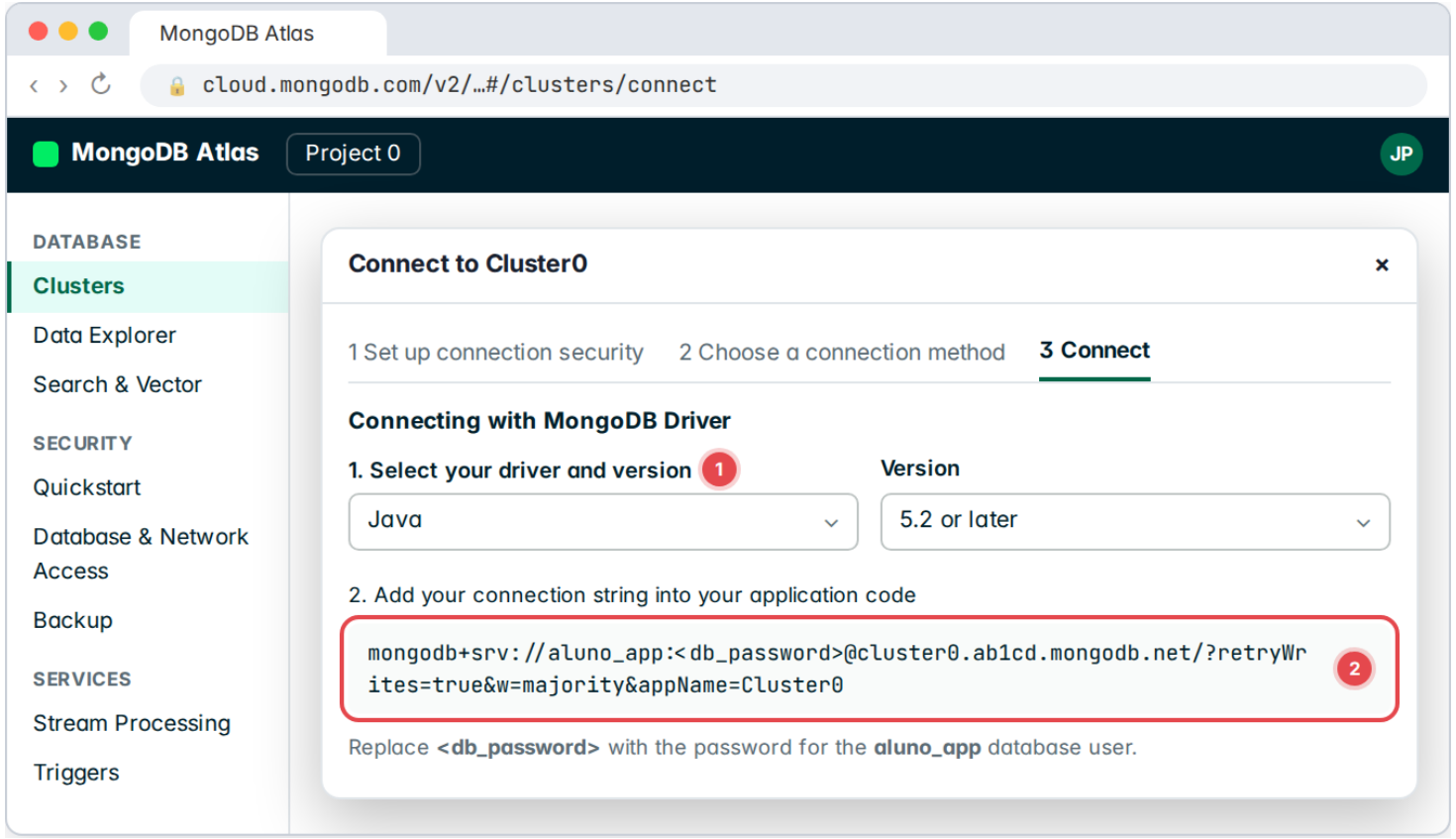
Observação

`0.0.0.0/0` significa *qualquer endereço*. A proteção passa a ser só o usuário e a senha do banco, por isso use uma senha forte e nunca a publique no GitHub. Em empresas, libera-se apenas o IP fixo do servidor.

04

Passo 3: copiar a connection string

Passo 3: copiar a connection string



Em **Clusters → Connect → Drivers**, escolha **Java** e copie a *connection string*.

Passo 3: copiar a connection string

1. Escolha o driver **Java** (o Spring Data MongoDB usa o driver oficial).
2. Copie a URL e troque `` `pela senha do usuário (sem os sinais` ``).

05

Passo 4: configurar o Spring Boot

Passo 4: configurar o Spring Boot

pom.xml

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
```

Passo 4: configurar o Spring Boot

Há duas formas de configurar a conexão. A **forma direta** usa a propriedade nativa do Spring Boot:

application.properties (forma direta)

```
# Spring Boot 4.x
spring.mongodb.uri=${MONGODB_URI:mongodb://localhost:27017/aula}
spring.mongodb.database=${MONGODB_DATABASE:aula}

# Spring Boot 3.x (projetos mais antigos, como BD_RestAPI_MongoDB)
# spring.data.mongodb.uri=...
# spring.data.mongodb.database=...
```

Passo 4: configurar o Spring Boot

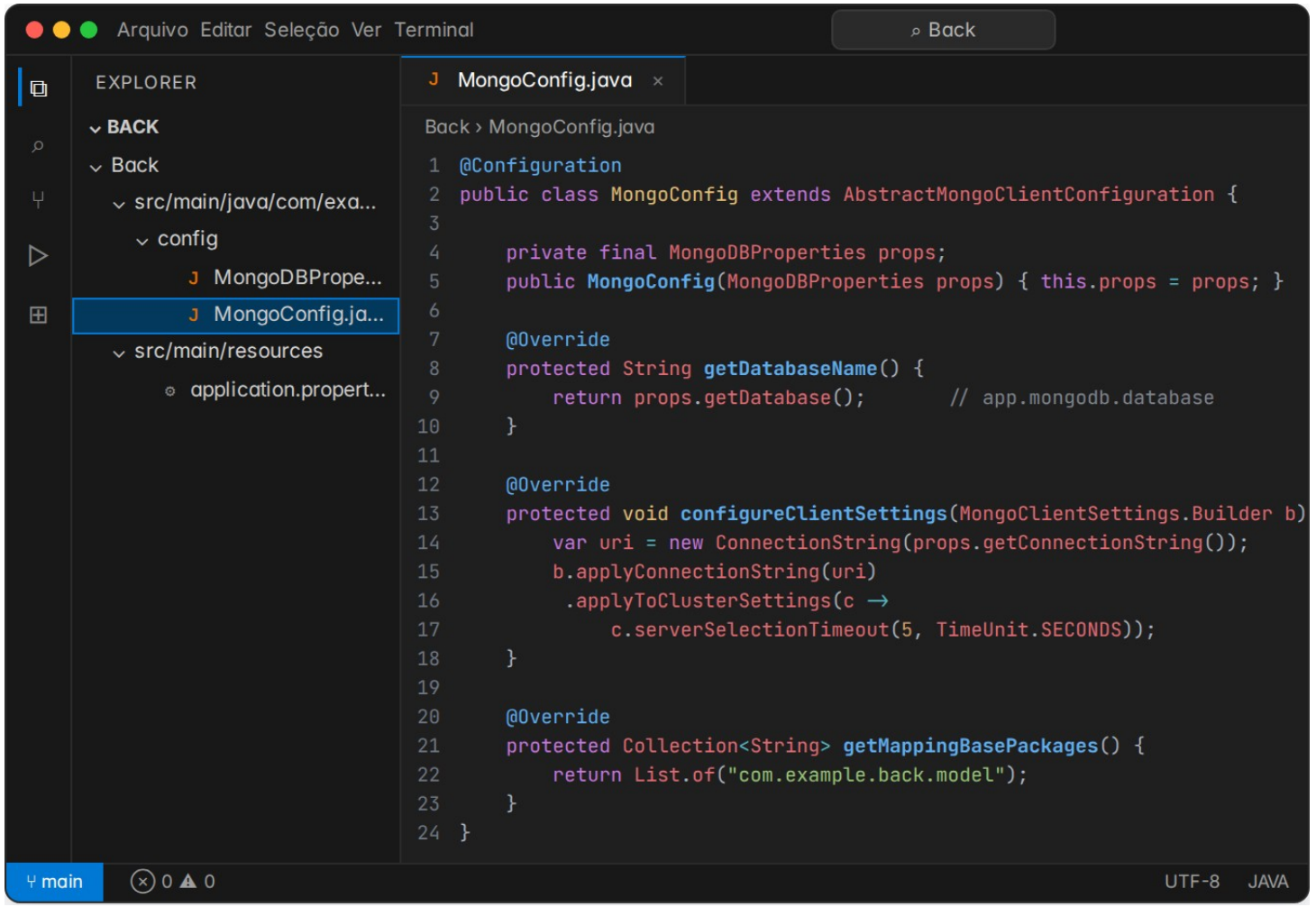
Os projetos `FullStack` usam a **forma explícita**: propriedades próprias (`app.mongodb.*`) lidas por uma classe `MongoDBProperties` e uma `MongoConfig` que cria o cliente. Assim a configuração não depende do nome da propriedade em cada versão do Boot e ainda dá para definir um *timeout* curto:

05

Passo 4: configurar o Spring Boot

Aula 07 · Back-end

Passo 4: configurar o Spring Boot



`MongoConfig` do projeto `Exemplo_Integracao`: conecta usando os valores do `application.properties`.

Passo 4: configurar o Spring Boot

application.properties (forma explícita)

```
app.mongodb.connection-string=${MONGODB_URI:mongodb://localhost:27017}
app.mongodb.database=${MONGODB_DATABASE:exemplo_integracao}
```

06

Passo 5: documento e repository

Passo 5: documento e repository

model/Aluno.java

```java
@Document(collection = "alunos")
public class Aluno {
    @Id
    private String id;      // ObjectId vira String
    private String nome;
    private String email;
    private String curso;

    public Aluno() { }
    // getters e setters
}
```

repository/AlunoRepository.java

```java
public interface AlunoRepository
        extends MongoRepository<Aluno, String> {

    List<Aluno>
    findByNomeContainingIgnoreCaseOrderByNomeAsc(
        String nome);

    boolean existsByEmail(String email);
}
```

Passo 5: documento e repository

Repare como é parecido com JPA: `@Document` no lugar de `@Entity`, `MongoRepository` no lugar de `JpaRepository`, e o `id` é `String`. Os **métodos derivados** funcionam igual. Não existe `ddl-auto`: o banco e a coleção são criados **no primeiro `save`**.

06

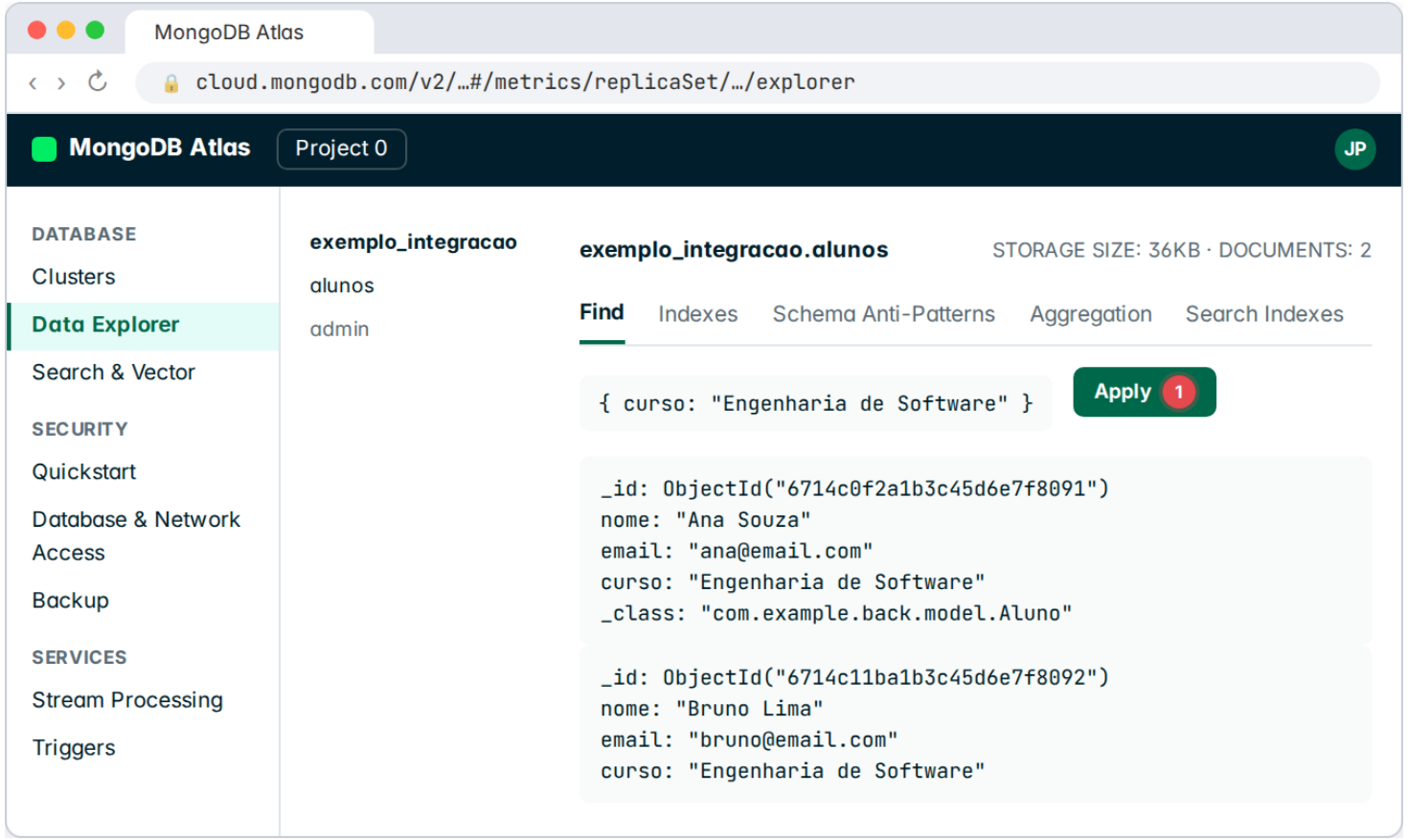
Passo 5: documento e repository

Aula 07 · Back-end

07

Passo 6: ver os documentos no Atlas

Passo 6: ver os documentos no Atlas



Data Explorer: banco, coleção e documentos gravados pela API, com filtro de busca.

Passo 6: ver os documentos no Atlas

Erro comum

`Timed out after 5000 ms while waiting to connect`: o IP não está liberado no *Network Access*, ou o cluster está pausado. `bad auth : authentication failed`: usuário ou senha errados (lembre de tirar os `). **MongoSocketReadException`** em rede de faculdade: algumas redes bloqueiam a porta 27017; teste com o roteador do celular.

Passo 6: ver os documentos no Atlas

Dica

Além do painel web, instale o **MongoDB for VS Code** ou o **MongoDB Compass**: cole a mesma *connection string* e navegue pelas coleções direto do seu computador.

08

Aprofundando: modelagem e consultas no MongoDB

Embutir x referenciar

Embutir (documento dentro de documento)

- Pedido com a lista de itens dentro
- Uma leitura traz tudo
- Bom quando os dados são lidos juntos e não crescem sem limite

vs

Referenciar (guardar o id)

- Aluno guarda o `cursoId`
- Precisa de duas consultas ou `$lookup`
- Bom quando o dado é compartilhado ou muito grande

Aprofundando: modelagem e consultas no MongoDB

Documento com lista embutida

```java
@Document(collection = "pedidos")
public class Pedido {
    @Id private String id;
    private String cliente;
    private List<Item> itens;            // embutido: vira array no documento
    private Instant criadoEm = Instant.now();
}

public record Item(String produto, int quantidade, BigDecimal preco) { }
```

Aprofundando: modelagem e consultas no MongoDB

Como fica no Atlas

```
{
  "_id": { "$oid": "6714c0f2a1b3c45d6e7f8091" },
  "cliente": "Ana",
  "itens": [
    { "produto": "Espresso", "quantidade": 2, "preco": 7.5 },
    { "produto": "Pão de queijo", "quantidade": 1, "preco": 5.0 }
  ],
  "criadoEm": { "$date": "2026-11-03T13:05:00Z" }
}
```

Consultas com MongoTemplate

Quando o nome do método derivado ficaria grande demais

```
@Service
public class RelatorioService {
    private final MongoTemplate mongo;
    public RelatorioService(MongoTemplate mongo) { this.mongo = mongo; }

    public List<Aluno> buscar(String curso, String trechoNome) {
        Query q = new Query()
            .addCriteria(Criteria.where("curso").is(curso))
            .addCriteria(Criteria.where("nome").regex(trechoNome, "i"))
            .with(Sort.by("nome")).limit(20);
        return mongo.find(q, Aluno.class);
    }
}
```

Aprofundando: modelagem e consultas no MongoDB

Agregação

```
// quantos alunos por curso?
Aggregation agg = Aggregation.newAggregation(
    Aggregation.group("curso").count().as("total"),
    Aggregation.sort(Sort.Direction.DESC, "total"));
List<Document> r = mongo.aggregate(agg, "alunos", Document.class).getMappedResults();
// [{ "_id": "Engenharia de Software", "total": 42 }, ...]
```

Índices

Sem índice, cada busca lê a coleção inteira. Com `@Indexed` o Spring cria o índice ao iniciar (ative `spring.data.mongodb.auto-index-creation=true`) ou você cria pela aba **Indexes** do Data Explorer. Índice único em e-mail também impede cadastros duplicados.

model/Aluno.java

```java
@Document(collection = "alunos")
public class Aluno {
    @Id private String id;
    @Indexed(unique = true) private String email;
    @Indexed private String curso;
}
```

Anatomia do ObjectId (12 bytes)

4 bytes: data e hora

Segundos desde 1970: o `_id` já diz quando o documento foi criado.

5 bytes: aleatórios

Únicos por máquina/processo, gerados pelo driver.

3 bytes: contador

Incrementa a cada novo id, evitando repetição no mesmo segundo.

Consequência

Ids gerados no cliente, sem consultar o banco, e ordenáveis por data.

Aprofundando: modelagem e consultas no MongoDB

Plano Free do Atlas	Limite aproximado
Armazenamento	512 MB
Conexões simultâneas	até 500
Clusters gratuitos por projeto	1
Backup automático	Não incluso
Inatividade prolongada	O cluster pode ser pausado; retome pelo painel

Aprofundando: modelagem e consultas no MongoDB

Dica

Prefira o MongoDB quando cada registro varia de formato ou quando você lê sempre o "documento inteiro" (um pedido com itens, um formulário). Prefira o Supabase quando há muitas relações e relatórios com `JOIN` e somas.

09

Quiz de revisão

Pergunta

Qual o equivalente a uma tabela no MongoDB?

A Documento

B Coleção

C Campo

D Cluster

Resposta

Qual o equivalente a uma tabela no MongoDB?

A Documento

✓ Coleção

C Campo

D Cluster

Por quê: Coleção agrupa documentos; documento equivale a uma linha.

Quiz · questão 2 de 5

Pergunta

Por que liberar `0.0.0.0/0` no Network Access do Atlas para o deploy?

A Para acelerar consultas

B O Render gratuito não tem IP fixo

C É obrigatório para o Compass

D Para criar o banco

Resposta

Por que liberar `0.0.0.0/0` no Network Access do Atlas para o deploy?

A Para acelerar consultas

✓ O Render gratuito não tem IP fixo

C É obrigatório para o Compass

D Para criar o banco

Por quê: A proteção passa a ser só o usuário e a senha do banco.

Pergunta

Quando o banco e a coleção são criados no Atlas?

A Ao criar o cluster

B No primeiro save da aplicação

C Ao rodar o Dockerfile

D Manualmente, sempre

Resposta

Quando o banco e a coleção são criados no Atlas?

A Ao criar o cluster

✓ No primeiro save da aplicação

C Ao rodar o Dockerfile

D Manualmente, sempre

Por quê: Não existe ddl-auto no MongoDB.

Pergunta

Qual o tipo do campo `@Id` normalmente usado com MongoRepository?

A Long

B String

C UUID

D int

Resposta

Qual o tipo do campo `@Id` normalmente usado com MongoRepository?

A Long

✓ String

C UUID

D int

Por quê: O ObjectId é convertido para String.

Pergunta

Erro `bad auth : authentication failed` indica:

A. IP não liberado

B. Usuário ou senha do banco errados

C. Cluster pausado

D. Falta o driver

Resposta

Erro `bad auth : authentication failed` indica:

A IP não liberado

✓ Usuário ou senha do banco errados

C Cluster pausado

D Falta o driver

Por quê: Lembre de tirar os sinais do .

Fechamento

Resumo da aula

- ✓ MongoDB guarda **documentos** JSON em **coleções**; o `_id` é gerado automaticamente.
- ✓ No Atlas: cluster **Free** → **usuário do banco** → **Network Access 0.0.0.0/0** → **Connect** → **Drivers** → **Java**.
- ✓ Dependência `spring-boot-starter-data-mongodb`; propriedade `spring.mongodb.uri` (Boot 4) ou configuração explícita com `MongoConfig`.
- ✓ `@Document` + `MongoRepository`; banco e coleção nascem no primeiro `save`.
- ✓ Confira os dados no **Data Explorer**.

Exercícios

1. Crie um cluster Free e conecte o projeto `Exemplo_Integracao/Back`.

2. Adicione o campo `List disciplinas` ao `Aluno` e veja o documento no Data Explorer.

3. Crie o método `findByCurso` e um endpoint `GET /api/alunos/curso/{curso}`.

4. Explique por que o deploy no Render falha se o Network Access tiver só o seu IP.

Para ir além

- mongodb.com/docs/atlas
- docs.spring.io/spring-data/mongodb/reference
- Projetos `BD_RestAPI_MongoDB` e `FullStack/AraCoffee/Back`

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 07?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web